Testing with input space partitioning

We are testing a `csv2json` program written in Rust that we found on GitHub

Specification of the Program Under Test

`CSV` (Comma Separated Values) files are used for representing data in spreadsheet programs. According to RFC 4180: 1. Each record is located on a separate line, delimited by a line break (CRLF). For example:

```
aaa,bbb,ccc CRLF
zzz,yyy,xxx CRLF
```

2. The last record in the file may or may not have an ending line break. For example:

```
aaa,bbb,ccc CRLF
zzz,yyy,xxx
```

3. There maybe an optional header line appearing as the first line of the file with the same format as normal record lines. This header will contain names corresponding to the fields in the file and should contain the same number of fields as the records in the rest of the file (the presence or absence of the header line should be indicated via the optional "header" parameter of this MIME type). For example:

```
field_name,field_name,field_name CRLF
aaa,bbb,ccc CRLF
zzz,yyy,xxx CRLF
```

4. Within the header and each record, there may be one or more fields, separated by commas. Each line should contain the same number of fields throughout the file. Spaces are considered part of a field and should not be ignored. The last field in the record must not be followed by a comma. For example:

```
aaa,bbb,ccc
```

5. Each field may or may not be enclosed in double quotes (however some programs, such as Microsoft Excel, do not use double quotes at all). If fields are not enclosed with double quotes, then double quotes may not appear inside the fields. For example:

```
"aaa","bbb","ccc" CRLF
zzz,yyy,xxx
```

6. Fields containing line breaks (CRLF), double quotes, and commas should be enclosed in double-quotes. For example:

```
"aaa","b CRLF
bb","ccc" CRLF
```

```
zzz,yyy,xxx
```

7. If double-quotes are used to enclose fields, then a double-quote appearing inside a field must be escaped by preceding it with another double quote. For example:

```
"aaa","b""bb","ccc"
```

`JSON` (JavaScript Object Notation) files are a commonly used serialized data exchange format over the internet. RFC 8259 defines it as: A JSON text is a sequence of tokens. The set of tokens includes six structural characters, strings, numbers, and three literal names.

These are the six structural characters:

```
begin-array     = ws %x5B ws  ; [ left square bracket

begin-object    = ws %x7B ws  ; { left curly bracket

end-array       = ws %x5D ws  ; ] right square bracket

end-object      = ws %x7D ws  ; } right curly bracket

name-separator  = ws %x3A ws  ; : colon

value-separator = ws %x2C ws  ; , comma
```

A JSON value MUST be an object, array, number, or string, or one of the following three literal names:

```
false
null
true
```

Input and Output relationship + Corner Cases

In addition to typical inputs, csv2json could encounter corner cases in the input that affect the JSON output, such as:

```
Empty file input: An Empty CSV file or files with only headers
Missing/Extra fields: More or less fields than the header specified
Empty values: Empty fields, partially or fully empty row
Array style headers: header such as name.first, name.last
```

Input Space Partitioning

Component and input identification

`csv2json` takes in a `CSV` (Comma Separated Value) file and optional flags and options:

```
$ csv2json --in <csv file> > <json file>
```

and outputs a formatted `JSON` file. The parameters are as follows:

`--delimiter (-d): Changes the CSV splitting character.`

`--dimensional-separator (-D): Creates nested JSON objects based on header syntax.`

`--arrays (-a): Breaks comma-separated string values into JSON arrays.`

`--remove-empty-strings: Strips empty string keys.`

`--remove-empty-objects: Strips empty JSON nodes.`

`--fold (-F): Folds row-by-row objects into a single object of arrays.`

`--jsonl: Outputs newline-delimited JSON Lines instead of a standard JSON array.`

Characteristics for partitioning the input space

To model this for combinatorial testing, we first partition the input domain into characteristics and mutually exclusive blocks (levels).

Characteristic 1: Input File State Blocks:

- Valid CSV Data
- Empty File
- Malformed CSV (e.g., unmatched quotes).
- Header only, missing file

Characteristic 2: Delimiter Option (`--delimiter, -d`) Blocks:

- Default (Comma)
- Custom (e.g., ‘`;`’ or ‘`:`’).

Characteristic 3: Dimensional Separator (`-D`) Blocks:

- None (Default)
- Custom (e.g., Dot .)

- Multidimensional (ex. `a.b.c`)
- Incorrect formats (`.a`, `a.`, `a..b`)
- Conflicting keys (`a` & `a.b` in same header)

Characteristic 4: Array Processing (`--arrays` or `-a` with `-D` to break items to arrays) Blocks: Enabled, Disabled.

Characteristic 5: Output Formatting (Mutually Exclusive) Blocks:

- Standard JSON
- Folded (`-F`)
- JSONL (`--jsonl`)

Characteristic 6: Remove empty strings/objects (`--remove-empty-strings`, `--remove-empty-objects`) Blocks:

- Use remove empty strings only
- Use remove empty blocks only
- Use none
- Use both Both commands happen to both arrays and objects (warned there may be undesired effects) Empty rows, partially empty rows

Characteristic 7: output destination (`--out-dir`, `--out-name`) Blocks:

- Default (stdout)
- `-out-dir`,
- `-out-dir` with `-out-name`

Characteristic 8: Column type conversion (`--boolean`, `-numeric`) Blocks:

- None
- `--numeric` only
- `--boolean`
- both